

Synchronous FIFO — Design Specification

Module Name: `syn_fifo` **File:** `syn_fifo.v` **Version:** 1.1 (corrected) **Type:** Single-clock (synchronous) FIFO, dual-port RAM based

1. Overview

`syn_fifo` is a parameterizable, single-clock-domain FIFO built around a dual-port RAM (`ram_dp_ar_aw`). One port is dedicated to writes, the other to reads, using independent write and read pointers and a status counter that tracks current occupancy.

This version corrects an off-by-one error in the original `full` flag and adds internal write/read qualification logic so the FIFO cannot silently overflow or underflow even if `wr_en`/`rd_en` are asserted when the FIFO is already full/empty.

2. Parameters

Parameter	Default	Description
<code>DATA_WIDTH</code>	8	Width of <code>data_in</code> / <code>data_out</code> , in bits
<code>ADDR_WIDTH</code>	8	Width of internal RAM address (pointer width)
<code>RAM_DEPTH</code>	<code>1 << ADDR_WIDTH</code> (derived, not user-set)	Number of storage locations (256 for default <code>ADDR_WIDTH</code>)

`RAM_DEPTH` is always a power of two, since it is derived from `ADDR_WIDTH`. This allows the write/read pointers to wrap correctly on overflow without explicit modulo logic.

3. Port List

Signal	Direction	Width	Description
<code>clk</code>	Input	1	System clock. All sequential logic is synchronous to the rising edge.
<code>rst</code>	Input	1	Active-high, asynchronous reset.
<code>wr_cs</code>	Input	1	Write chip-select. Must be asserted along with <code>wr_en</code> for a write to occur.
<code>rd_cs</code>	Input	1	Read chip-select. Must be asserted along with <code>rd_en</code> for a read to occur.
<code>wr_en</code>	Input	1	Write enable.
<code>rd_en</code>	Input	1	Read enable.
<code>data_in</code>	Input	<code>DATA_WIDTH</code>	Data to be written into the FIFO.
<code>data_out</code>	Output	<code>DATA_WIDTH</code>	Data read from the FIFO. Registered — valid one clock cycle after a qualified read request (see §6.3).
<code>full</code>	Output	1	Asserted when the FIFO is completely full (<code>status_cnt == RAM_DEPTH</code>).
<code>empty</code>	Output	1	Asserted when the FIFO is completely empty (<code>status_cnt == 0</code>).

4. Functional Description

4.1 Write Operation

A write occurs when `wr_cs && wr_en` are both asserted **and** the FIFO is not full. On a qualified write:

- `data_in` is written into the RAM at `wr_pointer`.
- `wr_pointer` increments by 1 (wraps automatically at `RAM_DEPTH`).
- `status_cnt` increments by 1 (unless a simultaneous qualified read also occurs — see §4.3).

If `wr_cs && wr_en` is asserted while `full` is already asserted, the write is **ignored**: no pointer increment, no RAM write side-effect change, no `status_cnt` change. This differs from the original code, which had no such protection.

4.2 Read Operation

A read occurs when `rd_cs && rd_en` are both asserted **and** the FIFO is not empty. On a qualified read:

- The RAM location addressed by `rd_pointer` is presented as `data_ram`, which is registered into `data_out` on the next clock edge.
- `rd_pointer` increments by 1 (wraps automatically at `RAM_DEPTH`).
- `status_cnt` decrements by 1 (unless a simultaneous qualified write also occurs — see §4.3).

If `rd_cs && rd_en` is asserted while `empty` is already asserted, the read is ignored: `rd_pointer` does not advance and `data_out` holds its previous value (no stale/garbage RAM read is latched).

4.3 Simultaneous Read and Write

If a qualified write and a qualified read occur on the same clock edge, one entry leaves and one entry enters in the same cycle, so `status_cnt` remains unchanged. Both pointers advance independently.

4.4 Reset

On `rst` (active-high, asynchronous):

- `wr_pointer`, `rd_pointer`, and `status_cnt` are cleared to 0.
- `data_out` is cleared to 0.
- `full` deasserts, `empty` asserts (since `status_cnt == 0`).

5. Status Flag Definitions

Flag	Condition	Meaning
<code>empty</code>	<code>status_cnt == 0</code>	No entries available to read.
<code>full</code>	<code>status_cnt == RAM_DEPTH</code>	No free space available to write. (corrected)

5.1 Correction Rationale

The original code defined:

```
verilog
assign full = (status_cnt == (RAM_DEPTH-1));
```

This asserted `full` one entry early — with `status_cnt` allowed to legally reach `RAM_DEPTH` per the counter's own write-guard (`status_cnt != RAM_DEPTH`), the FIFO's usable capacity was silently reduced from `RAM_DEPTH` to `RAM_DEPTH - 1` entries, and the discrepancy between the flag and the counter's true range was a latent design inconsistency.

The corrected version aligns the flag with the counter's actual range:

```
verilog

assign full = (status_cnt == RAM_DEPTH);
```

The FIFO now provides its full, advertised `RAM_DEPTH`-entry capacity.

6. Timing Characteristics

6.1 Write Timing

- `wr_pointer` and `status_cnt` update synchronously, one cycle after a qualified write request.
- Data is written into RAM combinationally on the same cycle the write is issued (RAM write is same-cycle; pointer/counter bookkeeping is registered).

6.2 Read Latency

- `data_out` reflects the RAM contents addressed by `rd_pointer` **one clock cycle after** a qualified `rd_en` assertion (registered read, consistent with synchronous dual-port block RAM behavior).
- Consumers must account for this 1-cycle latency; `data_out` is not combinationally valid in the same cycle as `rd_en`.

6.3 Back-to-Back Access

Because pointer and status updates are registered, back-to-back writes/reads are supported at one operation per clock cycle, as long as `full`/`empty` boundary conditions are respected.

7. Overflow / Underflow Protection

Condition	Original Behavior	Corrected Behavior
<code>wr_en</code> asserted while <code>full</code>	Not explicitly blocked; relied entirely on external controller discipline	Internally blocked — no pointer/counter/RAM side-effect change
<code>rd_en</code> asserted while <code>empty</code>	<code>rd_pointer</code> would advance past <code>wr_pointer</code> , and <code>data_out</code> would latch stale/undefined RAM contents with no error indication	Internally blocked — <code>rd_pointer</code> holds, <code>data_out</code> retains last valid value

This makes the FIFO self-protecting: even if the integrating design violates the `full`/`empty` handshake, the FIFO will not corrupt its internal pointers or silently return garbage data.

Design note: Self-protection adds a small amount of combinational qualification logic (`wr_valid`, `rd_valid`) on the enable paths. If area/timing is extremely constrained and the integrating logic is already verified to strictly honor `full`/`empty`, this guard can be omitted — but it is recommended to keep it for robustness during integration and verification.

8. Sub-module Dependency

`syn_fifo` instantiates a synchronous dual-port RAM:

```
ram_dp_ar_aw #(DATA_WIDTH, ADDR_WIDTH) DP_RAM (...)
```

Port	Direction	Description
<code>address_0</code>	Input	Write-port address (driven by <code>wr_pointer</code>)
<code>data_0</code>	Input	Write-port data (driven by <code>data_in</code>)
<code>cs_0</code>	Input	Write-port chip select (<code>wr_cs</code>)
<code>we_0</code>	Input	Write-port write-enable (<code>wr_en</code>)
<code>oe_0</code>	Input	Write-port output-enable (tied low — write-only use)
<code>address_1</code>	Input	Read-port address (driven by <code>rd_pointer</code>)
<code>data_1</code>	Output	Read-port data (feeds <code>data_ram</code>)
<code>cs_1</code>	Input	Read-port chip select (<code>rd_cs</code>)
<code>we_1</code>	Input	Read-port write-enable (tied low — read-only use)
<code>oe_1</code>	Input	Read-port output-enable (<code>rd_en</code>)

This RAM module is **not** included in this specification; it is assumed to be a standard synchronous dual-port RAM primitive with independent read/write address/control per port.

9. Known Limitations

- **Single clock domain only.** This design is not suitable for crossing clock domains; use a dual-clock (asynchronous) FIFO with Gray-coded pointers and proper CDC synchronizers for that use case.
- **No almost-full / almost-empty / programmable threshold flags.** Only binary `full`/`empty` are provided. If watermark flags are needed, `status_cnt` can be compared against a threshold externally or added as additional outputs.
- **RAM_DEPTH must be a power of two**, as it is derived from `ADDR_WIDTH`. Non-power-of-two depths would require additional wrap logic and are out of scope for this module.